

アジャイルワーク2

ニューラルネットワーク 重みデータ圧縮に関する実験計画書

学籍番号
氏名

25G1017
岩澤大心

作成日：2026年10月9日

提出期限：2026年10月16日（金）

目次

1 実験の目的	1
1.1 データ圧縮が必要な理由	1
1.2 検証する仮説	1
2 実験の理論	1
2.1 圧縮方式の調査と選択	1
2.2 ネットワークのデータ量	2
2.3 閾値による枝刈り	2
2.4 疎行列の格納方式	3
2.5 8 bit量子化	3
2.6 評価指標	3
3 実験方法	4
3.1 実験条件	4
3.2 実験条件の一覧	4
3.3 プログラムの実装手順	5
3.4 性能確認方法	6
3.5 方式と閾値の選択基準	6
3.6 結果の記録様式	7
4 予想される結果と問題発生時の対応	7
5 実施予定	7
6 再現性のために保存する情報	8
提出前チェックリスト	9

1 実験の目的

本実験の目的は、カラーチャートの読み取り値を補正するニューラルネットワークについて、Arduino上で使用する重みデータを圧縮し、限られたメモリ内で推論精度を保ったまま隠れ層の次元数を拡大できる条件を明らかにすることである。

1.1 データ圧縮が必要な理由

対象ネットワークは入力3次元、隠れ層2層、出力3次元である。授業資料では、隠れ層60次元はコンパイル・実行が可能である一方、70次元ではメモリ不足によるコンパイルエラーが確認されている。隠れ層を大きくすると表現能力の向上が期待できるが、第2層の重み数は次元数の二乗で増加するため、32 bit浮動小数点数のまま全要素を保存する方法には限界がある。

したがって、Arduinoの限られたメモリに、より大きな隠れ層を持つモデルを搭載するには、推論への影響が小さい情報を削減し、重みデータを小さくする必要がある。ただし、容量だけを小さくしても補正精度や実行速度が大きく悪化すれば実用にならないため、データ量、精度、速度を同時に測定し、圧縮方式を選ぶ必要がある。

そこで、L1正則化によって絶対値が小さくなった重みを0へ置き換える枝刈りと、非ゼロ要素だけを保持するCSR (Compressed Sparse Row) 形式を主方式として検討する。比較方式として8 bit整数への量子化も評価する。隠れ層40, 50, 60次元において、圧縮前後のモデル容量、Arduinoへの搭載可否、推論時間、平均二乗誤差 (MSE) を同一条件で比較し、メモリ削減量と推論品質のトレードオフを定量化する。

1.2 検証する仮説

1. L1正則化後の絶対値が小さい重みは推論への寄与が小さいため、適切な閾値で0化してもMSEの増加は小さい。
2. 十分な疎性が得られれば、CSR形式は列番号等の付加情報を含めても密行列より小さくなる。
3. 8 bit量子化は疎性に依存せず約4分の1まで重みを削減できるが、量子化誤差によりMSEが増える可能性がある。

2 実験の理論

2.1 圧縮方式の調査と選択

適用候補として、絶対値の小さい重みを0にする枝刈り、COO・CSR等の疎行列形式、重みを低ビット整数で表す量子化、および枝刈りと量子化を組み合わせるHybrid方式を先行研究[2, 3, 5]に基づいて調査する。各方式について、重み1要素当たりのデータ量、索引やスケール等の付加情報、Arduino上での行列ベクトル積の実装可能性、期待される計算量、推論誤差を比較する。

本実験では、L1正則化による係数縮小[1]を利用でき、行単位の積和演算にも適する枝刈り+CSRを主方式として選ぶ。一方、十分な疎性が得られない場合にも一定の削減効果を見込めるINT8量子化を比較方式として選ぶ。計画段階では一方に確定せず、両方式を同一条件で実装・測定し、容量、MSE、推論時間の選択基準に従って最終方式を決定する。どちらか一方に決められない場合は、両方式の結果と適用条件を記録する。

2.2 ネットワークのデータ量

隠れ層の次元数を H とすると、各層の重み行列は $H \times 3$, $H \times H$, $3 \times H$ である。順伝播は

$$\mathbf{h}_1 = \text{ReLU}(W_1 \mathbf{x} + \mathbf{b}_1), \quad (1)$$

$$\mathbf{h}_2 = \text{ReLU}(W_2 \mathbf{h}_1 + \mathbf{b}_2), \quad (2)$$

$$\mathbf{y} = W_3 \mathbf{h}_2 + \mathbf{b}_3 \quad (3)$$

によって計算される。パラメータ数は

$$N_{\text{weight}} = H^2 + 6H, \quad (4)$$

$$N_{\text{bias}} = 2H + 3, \quad (5)$$

$$N_{\text{total}} = H^2 + 8H + 3 \quad (6)$$

となる。すべてをfloat（1要素4 byte）で保持した場合のデータ量は $B_{\text{dense}} = 4N_{\text{total}}$ byteである。理論値を表1に示す。

Table 1: 密形式で保持した場合のパラメータ量（配列管理情報を除く）

H	重み数	バイアス数	合計要素数	FP32データ量 [byte]
40	1,840	83	1,923	7,692
50	2,800	103	2,903	11,612
60	3,960	123	4,083	16,332
70	5,320	143	5,463	21,852
100	10,600	203	10,803	43,212
200	41,200	403	41,603	166,412

この値以外にも、プログラム本体、入出力用配列、スタック等がArduinoのメモリを使用する。現行の60nn.inoでは、推論時に隠れ層出力`layer1[]`と`layer2[]`をローカルなFP32配列として確保するため、その作業領域は $8H$ byte（ $H = 60$ では480 byte）となる。さらに最大 6×6 画素の`imageData`が108 byteを使用する。したがって、理論値だけで搭載可否を判断せず、実際のコンパイル結果に表示されるフラッシュ使用量とRAM使用量を記録する。また、現行の`model_parameters.h`は修飾子なしのfloat配列を出力するため、全方式で`const`等の配置条件をそろえ、圧縮方式以外の差が生じないようにする。

2.3 閾値による枝刈り

学習後の重み w に閾値 τ を適用し、

$$\tilde{w} = \begin{cases} 0 & (|w| < \tau), \\ w & (|w| \geq \tau) \end{cases} \quad (7)$$

とする。0となった割合（疎性） S は、全重み数を N 、非ゼロ重み数を K として

$$S = 1 - \frac{K}{N} \quad (8)$$

で求める。本実験では $\tau = 0$ を非圧縮の基準とし、 $\tau \in \{0.001, 0.005, 0.01, 0.02, 0.05\}$ を候補にする。0.01は候補値の一つであり、固定値とはせず、MSEと容量の両方から選ぶ。

バイアスは要素数が少なく、0化による削減効果が小さいため、主実験ではFP32のまま保持する。これにより、圧縮による変化を重み行列に限定する。

2.4 疎行列の格納方式

$m \times n$ 行列に K 個の非ゼロ要素がある場合、COO形式は値、行番号、列番号を列挙する。一方、CSR形式は値配列、列番号配列、各行の開始位置を表す行ポインタ配列を使用する。CSRは疎行列ベクトル積で広く用いられる格納形式である[4]。 $H \leq 200$ では行・列番号をuint8_t, 行ポインタをuint16_tで表現できるため、理論データ量は概ね

$$B_{\text{COO}} = (4 + 1 + 1)K = 6K, \quad (9)$$

$$B_{\text{CSR}} = (4 + 1)K + 2(m + 1) \quad (10)$$

となる。3層分のCSR配列とFP32バイアスを合計すると

$$B_{\text{CSR,total}} = 5K + 12H + 24 \quad (11)$$

byteである。したがってCSRが密形式より小さくなる目安は

$$K < \frac{4H^2 + 20H - 12}{5} \quad (12)$$

となる。CSRでは行番号を要素ごとに保存しないため、本実験のような行列積に適している。ただし、C/C++コンパイラによるアラインメントや配列管理情報もあり得るため、最終的にはsizeofとコンパイル時のメモリ表示で実測する。CSRが密形式より大きくなる条件では採用しない。

CSR形式の行列ベクトル積は次のように実装する。

Listing 1: CSR形式による1層分の行列ベクトル積（概略）

```
for (uint16_t row = 0; row < rows; ++row) {
    float sum = bias[row];
    for (uint16_t p = row_ptr[row]; p < row_ptr[row + 1]; ++p) {
        sum += values[p] * input[col_idx[p]];
    }
    output[row] = sum;
}
```

2.5 8 bit量子化

比較方式では、整数演算向け量子化の先行研究[5]を参考に、層ごとに対称量子化を行う。層内の最大絶対値を $w_{\max}$ としてスケール $s = w_{\max}/127$ を求め、

$$q = \text{clip} \left(\text{round} \left(\frac{w}{s} \right), -127, 127 \right) \quad (13)$$

により重みをint8_tへ変換する。推論時は sq として使用する。重み1個を4 byteから1 byteへ削減でき、各層に4 byteのスケール値を追加する。3層分とFP32バイアスを合わせた理論データ量は $H^2 + 14H + 24$ byteである。バイアスはFP32のままとする。量子化は疎性が低い場合にも有効である一方、スケールの選び方や丸め誤差がMSEへ影響する。

2.6 評価指標

評価用カラーチャートの色数を N 、色 i の正解RGB値を t_i 、推論値を y_i とすると、MSEを

$$\text{MSE} = \frac{1}{3N} \sum_{i=1}^N \sum_{c \in \{R,G,B\}} (y_{i,c} - t_{i,c})^2 \quad (14)$$

とする．本実験で使用する 3×3 画像では $N = 9$ である．ニューラルネットワークへの入出力は $0.0-1.0$ へ正規化する一方，最終評価では従来どおり $0-255$ のRGB値へ戻してP3形式のPPM画像を生成し，配布済みのmse_calculator.htmlで評価チャートに対応する基準PPM画像と比較する．第3週の 3×3 チャートを使う場合の基準ファイルはreference_image.ppmである．全条件で同じ変換方法を使用する．圧縮前からの相対変化率は

$$\Delta_{\text{MSE}} = \frac{\text{MSE}_{\text{compressed}} - \text{MSE}_{\text{dense}}}{\text{MSE}_{\text{dense}}} \times 100 [\%] \quad (15)$$

で表す．また，モデルデータ削減率 R_B と圧縮比 C_B を

$$R_B = \left(1 - \frac{B_{\text{compressed}}}{B_{\text{dense}}}\right) \times 100 [\%], \quad (16)$$

$$C_B = \frac{B_{\text{dense}}}{B_{\text{compressed}}} \quad (17)$$

とする．

3 実験方法

3.1 実験条件

独立変数，従属変数，統制条件を表2に示す．

Table 2: 実験変数

分類	内容
独立変数	隠れ層次元 H (40, 50, 60)，圧縮方式 (密FP32, 枝刈り+CSR, INT8)，枝刈り閾値 τ
従属変数	MSE, MSE相対変化率, 疎性, モデルデータ量, 削減率, コンパイル可否, 実行可否, RAM・フラッシュ使用量, 推論時間
統制条件	学習データ, 学習済み重み, 評価チャート, 黒・白校正手順, SAMPLE_COUNT, RGB_FLASH_DELAY, RGB尺度, 測定治具, 照明, チャート位置, 紙の圧着, ソフトウェア版

学習条件は既存プログラムに合わせ，損失関数をMSE，最適化手法をAdam，学習率を 0.01 ，L1正則化係数を 10^{-6} ，学習回数を50,000 epochとする．圧縮の影響だけを比較するため，各 H について一度得た同一の学習済み重みからすべての圧縮条件を生成し，圧縮条件ごとに再学習は行わない．現行Notebookの学習データは， 3×3 の9色と， 6×6 で読み取った画像の上4行（ 6×4 の24色）を合わせた33組である．RGBを $0-255$ から $0.0-1.0$ へ正規化してtrain_l1_normalization.ipynbへ入力する．教師データは対応する2枚の基準PNG画像を最近傍法で 3×3 と 6×4 へ縮小して生成する．読取ミスを学習しないよう，元のPPM画像を目視確認してから学習データへ変換する．現行Notebookには乱数シードの固定処理がないため，モデル生成前にNumPyとPyTorchのシードを設定する．乱数シード，データ分割，Python・PyTorch・Arduino IDE・ボードコアのバージョンを実験記録へ残す．

3.2 実験条件の一覧

まずPC上で全閾値を事前評価し，Arduino実機へ載せるPrune-CSR条件を各 H につき1条件へ絞る．必須範囲の40, 50, 60次元が完了した後，まず非圧縮で限界を超える70次元を圧縮版で試し，容量に余裕があれば100次元を試す．200次元およびHybridは発展課題とし，必須条件の測定を優先する．

Table 3: 比較する方式

条件	重み表現	バイアス	目的
Dense-FP32	FP32密配列	FP32	圧縮前の基準
Pruned-Dense (PC確認)	枝刈り後のFP32密配列	FP32	枝刈り誤差とCSR実装誤差の切り分け
Prune-CSR	閾値枝刈り後のFP32値とCSR索引	FP32	小さい重みを除去した疎行列圧縮
INT8	層ごとの対称8 bit量子化	FP32	疎性に依存しない比較方式
Hybrid (意)	(任) 枝刈り後のINT8値とCSR索引	FP32	必須実験完了後の追加検討

3.3 プログラムの実装手順

1. **基準モデルの作成**： $H = 40, 50, 60$ について、 3×3 および 6×4 チャートから取得した同一の正規化済み学習データを用いて学習する。現行Notebookは $H = 40$ 、60nn.inoは $H = 60$ に固定されているため、Notebook側では単一の変数 H を3つのLinear層へ使用し、Arduino側のHIDDEN_SIZE1とHIDDEN_SIZE2も同じ値へ変更する。最終評価には、学習用データとは独立した評価用カラーチャートを教員から受け取って使用する。
2. **PC上での圧縮候補生成**：Pythonで各重みに閾値を適用し、疎性、CSR理論容量、圧縮後MSEを計算する。同じ重みからINT8版も生成する。
3. **ヘッダファイルの出力**：学習ノートブックが出力するmodel_parameters.hを基準とする。Dense-FP32では従来のweight_1[]-weight_3[]とbias_1[]-bias_3[]を出力する。Prune-CSRでは各層ごとにvalues[], col_idx[], row_ptr[]の3配列を出力し、それぞれ非ゼロ値、列番号、行開始位置を格納する。INT8ではint8_t重みと層ごとのスケールを出力する。各出力後に重み要素数が $3H$ 、 H^2 、 $3H$ 、バイアス要素数が H 、 H 、3であることを検査し、NotebookとArduinoの次元不一致をビルド前に検出する。
4. **Arduino推論処理の実装**：Dense、CSR、INT8の各行列ベクトル積関数を分離して実装する。既存のpredict()と同じく重みを出力ニューロン単位の行優先で参照し、隠れ層1・2だけにReLUを適用する。出力層にはReLUを適用しない。前処理は現行Arduino版と一致させる。normalizeInput()では、入力を $(\text{raw} - \text{black}) / (\text{white} - \text{black})$ で正規化して0-1へクリップする。outputTo255()では、出力を0-1へクリップし、255倍して四捨五入する。圧縮方式を変更しても、この前処理・後処理とバイアス加算順序は変更しない。
5. **単体確認**：PC上で枝刈り後の密行列とCSRから復元した行列が一致することを確認する。同じ固定入力に対するPruned-DenseとPrune-CSRの出力を比較した後、PC版とArduino版の各層出力を比較する。配列次元、CSR行ポインタ終端、整数のオーバーフロー、添字範囲を確認する。
6. **ビルド・実行確認**：各条件を同一ボード設定でクリーンビルドし、コンパイル可否、フラッシュ・RAM使用量を記録する。書き込み後、固定入力に対する出力と推論時間を確認する。

3.4 性能確認方法

実装後の性能は、Arduinoへの搭載可否、フラッシュ・RAM使用量、推論時間、MSEによって確認する。圧縮前のDense-FP32を基準として、Prune-CSRとINT8を同一条件で比較する。特にMSE測定では、照明、紙の圧着、位置ずれ、周辺光による変動を圧縮誤差と混同しないため、次の手順を用いる。

1. 学習に使用していない同一の評価用カラーチャートを全条件で使用する。
2. 非圧縮の基準には`batansyusei.ino`、ニューラルネットワーク版の基準には`60nn.ino`を用いる。両者ともD2で黒、白の順に校正し、D3で1画素ずつ読み取る。測定ブロックの開始時に黒を最小値、白を最大値として読み込む。モデルごとに同じ位置・手順で校正し、得られた値と時刻を記録する。
3. 1色のセンサ値は`SAMPLE_COUNT=10`回の平均とし、RGBの読取時間ずれを抑えるため、`RGB_FLASH_DELAY=10 ms`に固定する。センサ、光源、遮光、紙の固定治具、チャート位置を変えず、測定前のウォームアップ時間も一定にする。
4. チャートの9色を同じ順序で読み取り、各回について3×3のP3形式PPM画像を保存する。`mse_calculator.html`へ評価チャートに対応する基準PPM画像と読取画像を入力してMSEを記録する。
5. 各モデル・方式について同じチャートを10回測定する。測定順序はDenseと圧縮版を交互にし、時間変化の偏りを抑える。
6. 各回のMSEを保存し、平均 $\overline{\text{MSE}}$ 、標準偏差 s 、最小値、最大値を求める。圧縮版と基準版の各回の差も確認し、圧縮による差が基準版自身の測定ばらつきより大きいかを検討する。

推論時間は`micros()`で計測する。10回のウォームアップ後に100回連続実行し、合計時間を100で割った平均を用いる。シリアル出力時間は計測区間から除外する。

3.5 方式と閾値の選択基準

方式の選択は次の優先順位で行う。

1. **必須条件**：Arduinoでコンパイル、書き込み、推論が正常に完了する。
2. **品質条件**：暫定基準として、圧縮前に対する平均MSEの増加率を5%以下とする。さらに、その差が反復測定のばらつきと同程度かを確認する。
3. **容量条件**：品質条件を満たす候補のうち、実測モデルデータ量とRAM使用量が最小のものを優先する。
4. **速度条件**：推論時間がDense-FP32より20%を超えて増加する場合は、容量削減との利得を比較し、採否を再検討する。
5. **実装条件**：容量とMSEが同程度なら、配列数が少なく、誤添字やオーバーフローを防ぎやすい方式を選ぶ。

5%および20%は本計画で比較を行うための暫定基準であり、授業で指定された値ではない。最終判断では、Dense-FP32、Prune-CSR、INT8の「MSE-容量-速度」を図示し、いずれか一指標だけでなくPareto関係を確認する。CSRの索引を含む実容量が密形式以上なら、その閾値ではCSRを不採用とする。

3.6 結果の記録様式

各条件について、表4の項目を記録する．これにより理論値だけでなく、実機での搭載可否と推論品質を同時に比較できる．

Table 4: 実験結果記録表（測定時に記入）

H	方式	τ	疎性 [%]	データ量 [byte]	RAM [byte]	MSE 均±s	平 時間 [μ s]
40	Dense	0	0				
40	CSR						
40	INT8	–	–				
50	Dense	0	0				
50	CSR						
50	INT8	–	–				
60	Dense	0	0				
60	CSR						
60	INT8	–	–				

4 予想される結果と問題発生時の対応

L1正則化によって第2層を中心に絶対値の小さい重みが多くなり、適切な閾値ではMSEを大きく悪化させずにCSR容量を削減できると予想する．ただし疎性が低い場合、CSRは索引の分だけ Dense-FP32より大きくなる．INT8は一定の削減効果が得られるが、外れ値にスケールが支配されると小さい重みの量子化誤差が増える可能性がある．

閾値0.01で品質条件を満たさない場合は0.005, 0.001の順に閾値を下げる．CSRの容量削減が得られない場合はINT8を主候補とする．INT8のMSEが大きく悪化する場合は、16 bit量子化または層単位から行単位へのスケール変更を検討する．実測MSEのばらつきが圧縮前後の差より大きい場合は、固定治具と遮光を再確認し、反復回数を20回へ増やす．100次元以上が搭載できない場合でも、40–60次元の必須比較を完了させ、容量計算から障害原因を説明する．

5 実施予定

Table 5: 実験スケジュール

日付	内容
10月9–15日	方式調査、実験条件決定、計画書作成
10月16日	LaTeX原稿の最終確認、PDF化、manabaへ計画書を提出
実験第1日	次元数変更、枝刈り・CSR・INT8出力プログラムの実装
実験第2日	PC上での閾値探索、MSEと容量の事前評価
実験第3日	Arduino版の実装、固定入力による単体確認
実験第4日	実機でのメモリ・速度・MSE反復測定
実験第5日	集計、グラフ作成、方式の選択と考察

6 再現性のために保存する情報

実験で生成した学習済み重み、圧縮後ヘッダ、Python・Arduinoソース、測定CSVを条件ごとのフォルダへ保存する。各ファイルには H 、方式、閾値、乱数シードを含む名前を付ける。使用したArduinoボード名、コンパイラ・ボードコア・Python・PyTorchのバージョン、ビルドログ、黒・白の校正值、SAMPLE_COUNT、RGB_FLASH_DELAY、測定日時、評価チャート識別子、各回のPPM画像も記録し、同じ条件を再実行できるようにする。ニューラルネットワークを組み込む前のbatansyusei.inoと、現行の60nn.inoも比較用として変更せず別途保存する。

参考文献

- [1] R. Tibshirani, “Regression Shrinkage and Selection via the Lasso,” Journal of the Royal Statistical Society: Series B, vol. 58, no. 1, pp. 267–288, 1996.
- [2] S. Han, J. Pool, J. Tran, and W. J. Dally, “Learning Both Weights and Connections for Efficient Neural Networks,” Advances in Neural Information Processing Systems 28, pp. 1135–1143, 2015.
- [3] S. Han, H. Mao, and W. J. Dally, “Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding,” International Conference on Learning Representations (ICLR), 2016.
- [4] N. Bell and M. Garland, “Implementing Sparse Matrix-Vector Multiplication on Throughput-Oriented Processors,” Proceedings of the Conference on High Performance Computing Networking, Storage and Analysis, Article 18, pp. 1–11, 2009.
- [5] B. Jacob et al., “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 2704–2713, 2018.

提出前チェックリスト

- ☐ 表紙の学籍番号と氏名を本人の情報へ変更した.
- ☐ 使用するArduinoボード名と各ソフトウェアのバージョンを追記した.
- ☐ LaTeXからPDFを生成し, 数式・表・改ページ・文字化けを目視確認した.
- ☐ 未実施の測定結果を記入していないことを確認した.
- ☐ ファイル名とmanabaの提出欄を確認し, 2026年10月16日(金)までに提出した.